01 / THE PROJECT

Private requests.
Provable balances.

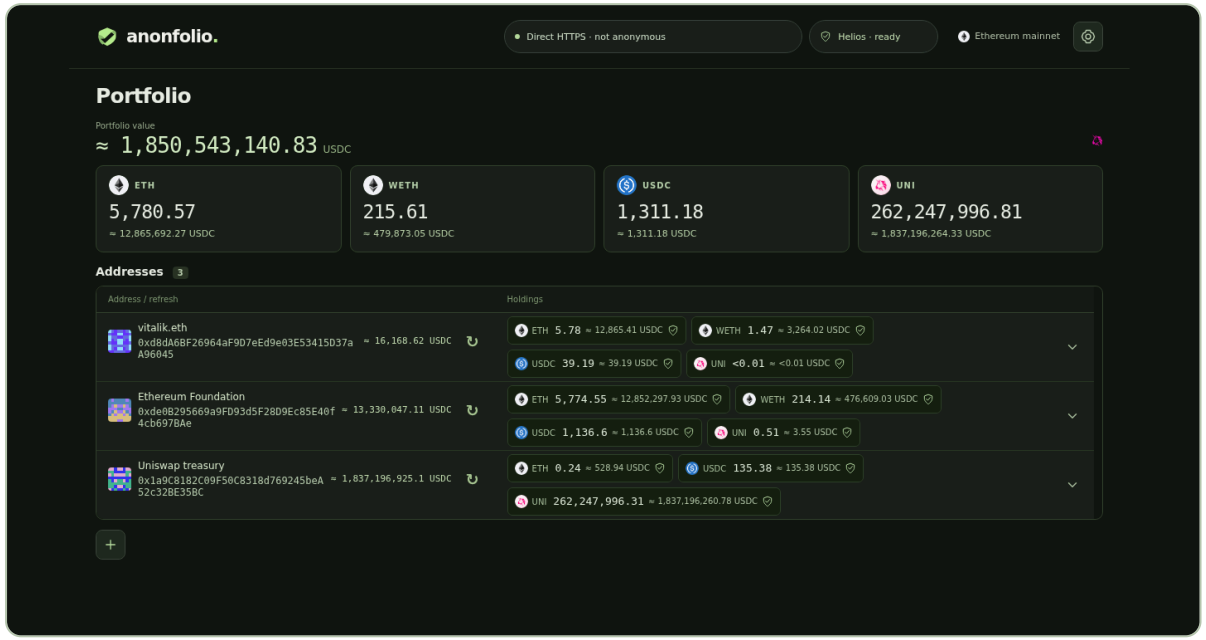
Anonfolio is a static Ethereum portfolio reader. It sends requests through Anon-RPC and Tor, then verifies balances inside the browser.

Ethereum mainnet | ETH · WETH · custom ERC-20s | No wallet connection or signing

> An RPC operator supplies the evidence. Your browser decides whether the result is valid.

02 / WHAT THE USER SEES

A portfolio, without connecting a wallet.



Public mainnet snapshot · 6 Nov 2026 · ETH, WETH, USDC and UNI · Helios-verified over direct HTTPS.

- Track named addresses and custom tokens with browser-local icons.
- Refresh one address; inspect timestamped verification logs and errors.
- See token totals and optional direct-pool price estimates.
- Reopen saved data offline. New reads start only when requested, except retries for an active failed read and initial reads for newly added addresses.

03 / THREE DIFFERENT GUARANTEES

Privacy, authenticity and availability are separate concerns.

Privacy

Tor routes requests away from the browser’s public IP. Each address gets its own isolation key.

`Anon-RPC + tor-js`

Authenticity

Consensus establishes a trusted state root. Proofs and local EVM execution bind the answer to it.

`Helios + Merkle proofs`

Availability

Gateways, Tor routes and providers can fail. Timeouts, retries and saved values keep the UI usable.

`Fail closed + local cache`

> The RPC still sees the requested address. Different isolation keys do not guarantee different Tor exit IPs.

04 / TECHNOLOGY MAP

Small UI. A substantial verification stack.

TypeScript + DOM

Native HTML/CSS, typed orchestration and in-place DOM updates. No frontend framework.

`TypeScript 5.9.3`

Bun + Vite

Locked dependencies, unit tests, development server and static production bundles.

`Bun baseline 1.3.14 · Vite 7.3.7`

Anon-RPC + Tor

A sandboxed, hash-pinned tor-js worker exposes private fetch over a KPS/WebRTC bridge.

`Harness 0.4.0 · tor-js 0.4.4`

WASM Helios + revm

Consensus light client, validated state, and local EVM execution in a dedicated worker.

`Reviewed Helios 0.12.0 · PR #813 fixes`

viem + EthereumJS

ABI/address utilities, exact unit formatting and additional account/storage trie checks.

`viem 2.38.0 · Trie 6.2.1`

Uniswap v3

Verified pool reads produce a 30-minute TWAP. One direct pool; bigint valuation arithmetic.

`Canonical mainnet factory · USDC default`

05 / PROCESSES AND CAPABILITIES

Two workers with different responsibilities.

BROWSER / APPLICATION ORIGIN

Main thread · UI + transport bridge

Watchlist, request queue, AbortControllers, progress and browser storage.

Helios worker

Shared consensus client, pinned-block sessions, local revm and oracle cache.

`postMessage ⇄ host`

NULL-ORIGIN IFRAME

tor-js worker

Private fetch and Tor state. No ambient access to the application’s DOM, cookies or keys.

Harness capabilities

The host bridges KPS/WebRTC, namespaced IndexedDB and logging. The Tor worker does not call WebRTC directly.

Outside the browser

KPS gateway → Tor network → execution RPC / beacon API.

The sandbox limits the privacy worker’s capabilities. It does not establish Ethereum consensus or verify balances.

06 / STARTING WITHOUT A PUBLIC RPC DEPENDENCY

Pin code first. Start the private route.

01 **Resolve the bundled Tor worker locally**

A narrow specifier provider answers worker hash and URL lookups from release metadata.

02 **Verify before executing**

The harness checks Keccak-256 of the worker bytes, then starts the sandbox.

03 **Connect through configured gateways**

An optional public directory mirror can seed Tor; Tor authenticates directory data.

04 **Initialize Helios on a requested read**

Download local WASM, check SHA-256, then bootstrap consensus from the selected checkpoint.

```
Release trust inputs

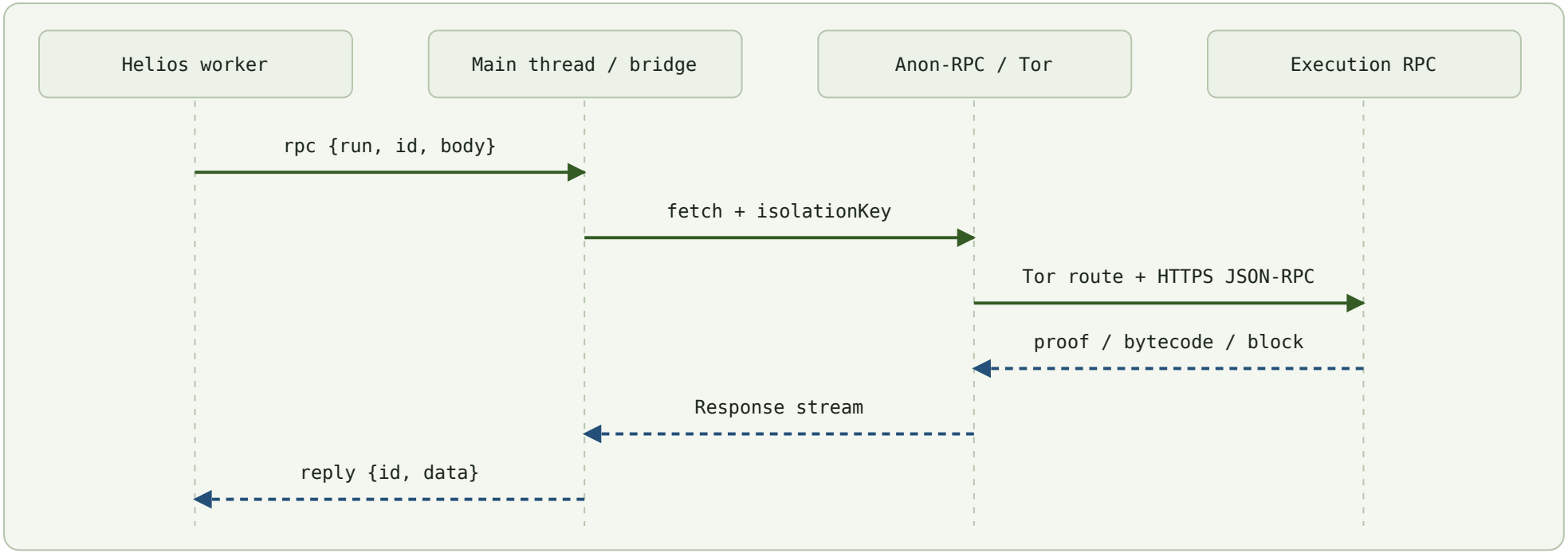
Tor worker digest   → Keccak-256
Helios WASM digest  → SHA-256
Mainnet checkpoint  → root + slot
Gateway identity    → certificate hash
```

> Page open starts transport startup. It does not automatically refresh saved balances.

Recovery exception: “Update Tor worker from PublicNode” explicitly performs direct metadata/download requests and changes the selected worker trust pin. No tracked addresses are included.

07 / ONE EXECUTION REQUEST, END TO END

Messages carry requests. Proofs carry trust.



Conceptual sequence, not a latency measurement.

`run` identifies the wallet session; `id` correlates an individual pending request.

Helios then emits `balance`, `price`, progress or failure events. The UI saves accepted results.

08 / WHY KPS AND WEBRTC ARE INVOLVED

A browser cannot simply open a Tor TCP socket.

Browser harness

KPS bridge exposes streams to the Tor worker.

→

WebRTC / DTLS

Browser transport to an `ip:port:certhash` endpoint.

→

KPS gateway

Pinned certificate identity; carries Tor transport traffic.

→

Tor circuits

Relays → exit → HTTPS provider connection.

Identity and isolation

KPS pins a peer certificate hash. Tor builds circuits over the provided transport. A random per-address `anonOptions.isolationKey` stays with balance/proof/oracle retries.

Separate service destinations

Execution requests go to configured mainnet RPC operators. Beacon light-client requests go to the pinned consensus API on a separate shared route.

The gateway is transport infrastructure, not the execution RPC. No balance request falls back to ordinary browser fetch in Tor mode.

09 / AUTHENTICATE THE BLOCK BEFORE ITS STATE

Helios anchors every refresh to finalized state.

Checkpoint

Independent mainnet beacon root + matching slot.

→

Consensus updates

Sync-committee signatures and finalized header proofs.

→

Execution block

Authenticated execution payload and validated block data.

→

Refresh snapshot

One block hash, state root, number and timestamp.

```
VerifiedBlock = { hash, stateRoot, number, timestamp }

ETH proof          ┐
ERC-20 execution   ┼─ all pinned to this refresh's block hash
Price oracle reads ┘
```

> Consensus is shared. Each wallet refresh selects its own finalized snapshot; different refreshes may select different blocks.

10 / NATIVE ETH VERIFICATION

ETH is an account proof, not a trusted balance string.

```
eth_getProof(
  walletAddress,
  [],
  { blockHash: verifiedBlock.hash }
)
```

```
stateRoot
↓ account-trie proof at keccak256(address)
[nonce, balance, storageRoot, codeHash]
```

01 **Bind the response to the requested wallet**

A valid proof for another address is rejected.

02 **Check the path against the authenticated root**

Wrong-block or modified account data cannot pass.

03 **Accept the raw bigint balance**

A zero value needs a valid inclusion or non-inclusion proof. A failed request is unavailable, never zero.

11 / EVERY ERC-20 FOLLOWS THE SAME PATH

Execute balanceOf locally, with proven dependencies.

ABI input

`0x70a08231` + padded wallet address.

→

Proven account

Authenticate code hash and storage root.

→

Verified code/state

Fetch code; verify its hash. Demand-load storage proofs.

→

Local revm

Run contract logic and decode one uint256 result.

Proxy-aware execution

Implementation contracts, delegated calls and additional storage are discovered during execution and verified through the same private route.

Display metadata stays explicit

The user supplies token symbols, decimals and icons. The app does not call `decimals()`. WETH is an ordinary ERC-20 included by default.

> The execution bridge permits proof, code and block requests. Remote `eth_call` is not an authoritative execution shortcut.

12 / VERIFIED PUBLIC ORACLE DATA

A 30-minute TWAP from one direct pool.

Uniswap v3 → USDC by default



observe([1800, 0]) · schematic window, not market data

```
meanTick = floor(ΔtickCumulative / 1800)
rawRatio ≈ 1.0001 ^ meanTick
holdingValue = rawBalance × orientedRatio
```

01 **Validate the direct pair**

Resolve through the canonical v3 factory; check pool factory, tokens and fee.

02 **Require history and liquidity**

30-minute observations plus sufficient current and harmonic virtual quote reserves.

03 **Use the balance’s verified block**

Oracle calls run locally over proven state. No multihop routes or external price API.

ETH and WETH exclusively use the WETH/USDC 0.05% pool with USDC selected. Missing direct pool → unpriced holding → partial portfolio total.

13 / PARALLEL WORK AND SHARED QUOTES

Parallel within an address.
Shared prices across addresses.

Wallet queue **A · active** → **B · queued**

At A’s block 2 balance jobs | 2 pricing jobs

Quote identity `base + quote + window + block + liquidity floor`

Different wallet sessions serialize through one verifier, keeping their transport contexts separate.

REGRESSION FIXTURE · SAME BLOCK & SETTINGS

10 addresses

ETH + WETH holdings reuse **one successful pool lookup**.

8 logical contract reads total

The first quote checks factory, pool identity and observations. Proof/code HTTP request counts vary. The next nine routes do no additional oracle reads.

A new finalized block needs a new quote. Caches are bounded and in memory; verifier restart or eviction requires fresh verification. Saved quotes remain historical display data.

14 / A BAD CONNECTION MUST NOT FABRICATE DATA

Preserve the last result. Explain the next attempt.

Proven zero

A valid proof returned zero. This is a successful result, not an empty UI placeholder.

Unavailable

No accepted amount exists. Show a reason and retry action; do not substitute zero.

Stale / saved

Keep the previous amount and provenance visible when refreshing fails or data are restored.

Persistence

localStorage stores balances, block identity/time, quotes, settings and errors. Web Locks serialize merges; newer finalized blocks win. Cross-tab updates start no reads.

Bounded recovery

Failed reads and transient price failures retry on the address’s route. Permanent “no qualifying pool” errors stay unpriced. Removal aborts active and queued work.

15 / STATIC DISTRIBUTION AND EXPLICIT MODES

The host serves files.
The browser performs verification.

Bun + Vite build

HTML, CSS, JS, worker bundles, WASM and local artwork.

→

Static HTTPS / IPFS

Complete `dist/` directory. Relative asset paths support gateways.

→

Service worker

Cache application assets for offline reopening, not RPC responses.

Selected mode	Transport	Who sees the browser IP?	Verification
Tor · default	Anon-RPC / tor-js	Connecting gateway; RPC sees Tor exit	Helios + local proofs/EVM
Direct HTTPS · explicit	Ordinary browser fetch	Execution and consensus providers	Same Helios verification

> No automatic Tor → HTTPS fallback. The selected mode is visible in the header.

Follow the boundaries in the source.

```
bun install --frozen-lockfile
bun run dev

bun run check
bunx playwright install chromium
bun run test:browser

bun run build
bun run serve
bun run test:static
```

UI + state

Refresh orchestration, persistence and rendering.

Private transport

Worker startup, pins, readiness and cancellation.

Verification bridge

Session messages, routing, WASM, proofs and local EVM.

TWAP + shared cache

Direct pools, exact arithmetic and quote reuse.

Adversarial unit tests

Wrong wallet/block, omitted storage, tampered code, privacy escapes.

Playwright browser suite

Cross-tab races, recovery, layout, offline and opt-in live tests.



Live HTTPS and Tor smoke tests are separate, opt-in checks and depend on external connectivity. Passing deterministic tests does not establish live Tor availability.

Technology marks belong to their owners. Tor and the Tor Onion logo are trademarks of The Tor Project. Anonfolio is not affiliated with or endorsed by The Tor Project. Artwork credits.